Forward Proxy Team: Test Plan Template

Name of Feature	<Feature Name>
NPLAN & ENG Epic	<Link of NPLAN> <Link of ENG Epic>
Product requirement doc	<Link of PRD>
Feature design doc	<Link of Design Doc>
Release Schedule	<R101/R102>
QE Members	<QE Name>
Dev Members	<DEV Name>
TestPlan Review by QE & Dev & PM	TBD / DONE Review date: <>
Test Rail Link	<Link of TestRail>
Testing Estimates (in days or weeks) **Manual + Automation**	Total:<> Manual:<> Automation: <>
Feature TOI link	
Version [Version to be updated each time Testplan is updated]	1.0(Increments of 1)

Test Scope

The scope is to give the complete test strategy for the delivery of a feature. All project members will have a **clear** understanding about what is tested and what is not.

- In test scope: High level What will be tested as part of this engagement

- Not in test scope: Something is not supposed to be tested.
  - Include
    - Partial coverage of features
    - Interoperability with third parties
    - Aspects of stress, solution

In test scope	Not in test scope

Setup diagram

High Level Setup Diagram

Test Requirements

Include information about the stack and setup where testing will take place along with availability status.

- Stack access
- MP/DP dependent services (VM or Stork)
- Tenants / Test Clients
- Load Balancers
- Tools used in Testing
- Enable feature in dogfoog.hippo.local (if needed)
  - Ref: Dogfood testing

Requirements	Availability Status

Feature Flags

List control flags, feature flags, group_vars, provisioner flags, USRA flags if applicable

Type	Flag Name	Default Status
URSA		
Ansible group_vars		

Provisioner		
WebUI		
staged_config		

Dependency features

The section should identify and capture if there are feature within the proxy that may be impacted or should be tested.
E.g: CTEP/IPS/LB/DLP/TSS/TokenProxy/AuthService/Steering methods(IPSec/GRE/ExplictProxy/NSClient)/Appliance

Regression Impact

This section should list down and capture areas, if the newly introduced features has a regression impact on the existing features.

Also include different proxy modules which needs to be tested with the new feature
Note: Regression should be identified on code change area and modify existing automation.

Acceptance Criteria.

The feature would be accepted if the below use cases are executed by the development and are setup to pass.

Resiliency of Service/Feature

Consider the ability of the service or feature to withstand and recover from failures, errors, or unexpected events. Ask yourself:

- What are the potential failure points in the service or feature?
- How will the service or feature behave in case of failure or error?
- Are there adequate error handling and recovery mechanisms in place?
- Can the service or feature recover from failures or errors without human intervention?
- Are there any single points of failure that could bring down the entire service or feature?

Test Cases

Breakdown all test cases into following Test Types(as applicable) and write a test in following test types.

S. No	Section	Test Categories(Type)	Service/ Component	Test Summary	Steps	Expected Result	Priority(P0/ P1/P2/P3)	Automatable (Yes /Nn)	Automated (Yes /Nn)	UI Case (Yes/Nn)	Arrived by QE (Yes /Nn)	Suggested by Dev (Yes/ No)	Derived by AI (Yes /No)
		Deployment											
		Component											
		Functional											
		UI											
		Negative											
		Component Scale											
		Performance											
		Security											
		NS Appliance Test (if any impact)											

Detailed Information about Test Categories

Test Types Detail

Deployment Test

Write the deployment validation cases necessary to be validated.

- Component Fresh deployment (Ansible / Stork)
- Ansible deployment of Ubuntu16 and Ubuntu20
- Version Upgrade (Ansible / Stork)
- Version Downgrade (Ansible / Stork)
- Deployment Idempotency
- with/out new group_vars

Component Test

Write the component validation cases necessary to be validated

- Component functioning
- Config files & database validation
- Config sync & Load
- Negative cases
- API CRUD
- Test with multiple tenants
- Test with multiple users/tunnels
- Service/Container validation
  - start/stop/restart/reboot
  - Pod scale up / down
  - Log files validation / rotation
  - nswatson / debug log → toggle
  - Service Health check
  - Service Monitoring
  - Sumo Logic Alerts
  - Service metric collection
  - Service crash/dump backtrace
  - caching

Component Scale Test

Component Scale test cases that are needed. Look at the requirements.

WebUI Test

- UI with Enable/Disable feature flags
- CRUD Operation (Single/Multiple Objects)
- Enable/Disable/Clone/Ordering of Object
- Link Navigation & Page Layout & alignments
- Error messages
- Unicode/Special Characters/Invalid Input/Empty String
- Required fields
- Audit Logs

End to End Functional Test

- Real World scenarios of Feature
- End to end functional test without Mockers
- Validation with dependent services
- SkopeIT events

Performance Test

Performance test cases needed to be covered have to be added here.

- Max HTTP/HTTPs connections
- Verify Latency and throughput
- Traffic from multiple users / tunnels / tenants
- Memory/CPU Utilisation on max load
- Monitor logs
- No service crash

Security Test

Does the feature need pen testing? Will negative scenarios create a security hole? Does it open new ports or create new attack surfaces/vectors?

Appliance Test (if any impact)

Actionable Items

- [ ] Test coverage in SRE Tool , MPAS/DPAS PDV needed?
- [ ] Feature applicable for Appliance ?
- [ ] Feature Automation needed?

TOI Links

Key Bugs/Tasks which needs special attention